

即网络包的收发处理基本不需要 Linux 内核的参与。DPDK 的设计思路如下。

用户空间驱动

为了能在用户态同网卡设备进行交互，DPDK 需要在用户态直接执行网卡驱动代码。其做法是将设备寄存器直接映射到应用自身的进程地址空间中，进而让 DPDK 的用户态驱动通过 MMIO 操作设备。

正如操作系统为应用程序的开发提供统一设备文件系统和 I/O 使用接口一样，Linux 提供了用户态驱动开发框架，即 UIO (Userspace I/O)。Linux 将 UIO 设备抽象为路径为 `/dev/uio [ X ]` 的设备文件。应用程序通过打开 UIO 设备文件获取设备的 I/O 空间和中断信息，同时自行决定如何操作和响应设备。

为了创建设备文件，UIO 驱动编写者需要在 Linux 系统中注册一个简单的 UIO 内核模块。代码片段 13.17 展示了 UIO 内核模块需要注册的结构体信息，其中，`mmap` 用于将设备的地址空间和端口空间映射到用户地址空间，`irqcontrol` 用于开启或屏蔽中断。UIO 驱动框架提供默认的读语义，`read` 会将用户空间驱动阻塞在等待中断上，当中断到来时 `read` 会立即返回，在返回值中表明一共发生了几次中断，UIO 用户驱动通常不使用 `write` 接口，而是从已经经过 `mmap` 处理的内存区间直接与设备交换数据。

代码片段 13.17 Linux 中 `uio_info` 结构体的定义

```

1 //include/linux/uio_driver.h
2
3 struct uio_info {
4     struct uio_device * uio_dev;
5     const char * name;           // UIO 设备名称
6     const char * version;       // UIO 设备版本号
7
8     // 用于映射到用户空间的内存和端口
9     struct uio_mem mem[MAX_UIO_MAPS];
10    struct uio_port port[MAX_UIO_PORT_REGIONS];
11
12    // 中断号
13    long irq;
14    // 中断处理函数，通常只负责确认中断，而不做数据处理
15    irqreturn_t (* handler) (int irq, struct uio_info *dev_info);
16
17    // 和用户空间驱动代码交互的接口
18    int (* mmap) (struct uio_info *info, struct vm_area_struct *vma);
19    int (* open) (struct uio_info *info, struct inode *inode);
20    int (* release) (struct uio_info *info, struct inode *inode);
21    int (* irqcontrol) (struct uio_info *info, s32 irq_on);
22 };

```

轮询模式驱动

内核态驱动的中断响应可以通过硬件机制实现，但用户态驱动的中断响应就没有那么直接了。中断发生时，CPU 首先跳转到 UIO 内核模块提前注册的中断处理函数处，该

函数通知操作系统的调度器，在中断处理函数返回后通过调度器进一步唤醒 UIO 用户态驱动正在等待中断的线程（该线程被阻塞而默认处于睡眠状态，直到中断发生）。整个过程涉及的上下文切换次数更多、时延更长，并不能达到预想中高性能处理网络包的效果。

为了避免中断带来的开销，DPDK 使用基于轮询模式的驱动程序（Poll Mode Drivers, PMD）。轮询模式驱动的处理思路和 NAPI 十分相似。如图 13.22 所示，基于 PMD 的网卡驱动首先通过 UIO 框架将设备的寄存器空间和 DMA 描述符映射到用户空间，随后开启中断。当网卡产生中断时，CPU 跳转到 PMD 网卡驱动注册的 UIO 内核态中断处理函数，该函数随即唤醒 PMD 被阻塞的处理线程，并通知网卡中断事件的到来。PMD 的处理线程被唤醒后，将立即屏蔽网卡中断，然后以轮询的方式在用户态操作网卡描述符并读写对应的 DMA 缓冲区，实现低延迟的网络数据包处理。不再有数据包到来后，PMD 线程停止收包，并再次将中断模式打开。

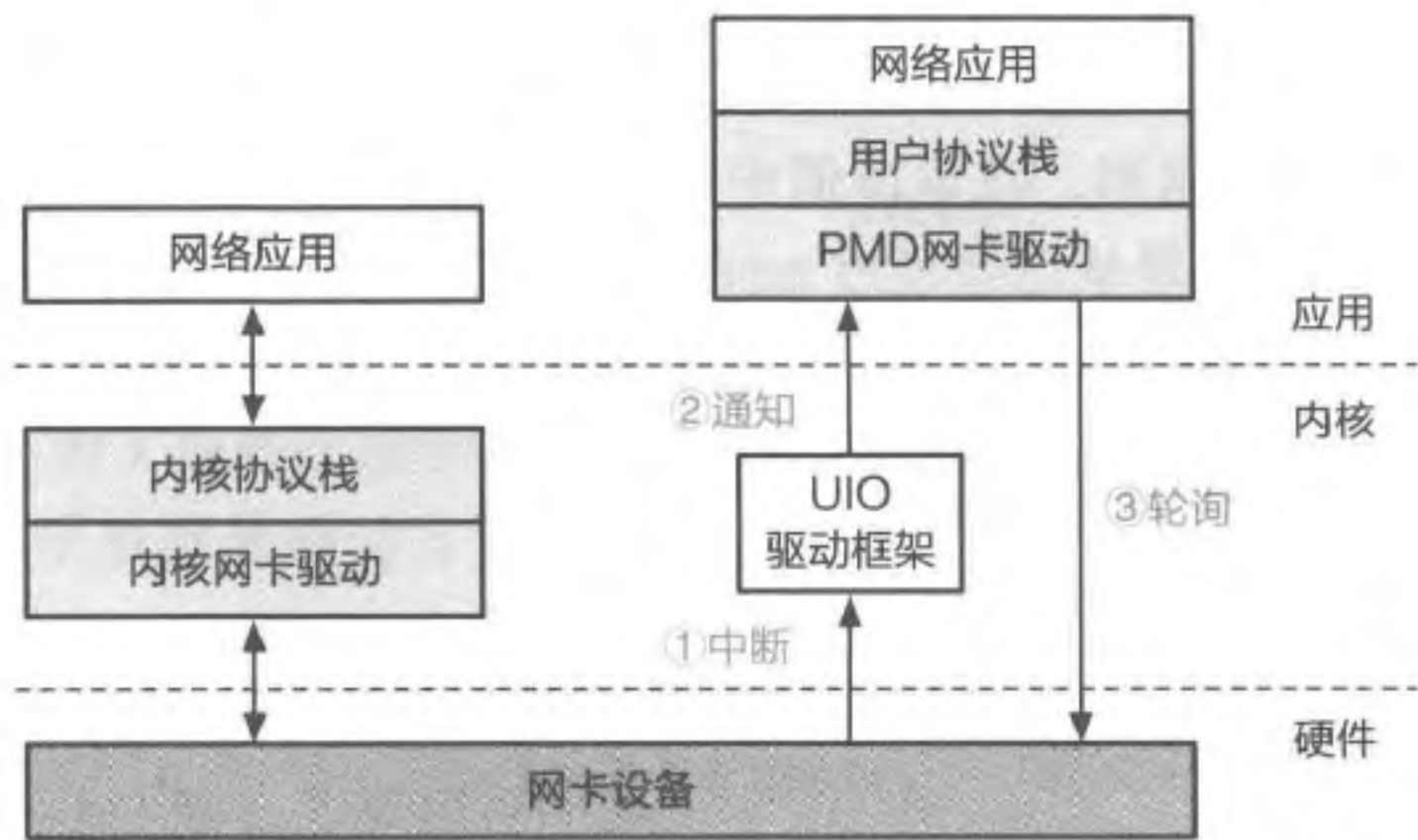


图 13.22 传统 Linux 内核网络 I/O 与基于 UIO 框架的 DPDK 网络层次示意图

13.5 案例分析：Android 操作系统的硬件抽象层

Android 操作系统是由谷歌公司和开放手机联盟共同主导开发的基于 Linux 内核的操作系统，主要面向移动端智能设备。Android 操作系统在 Linux 内核之上增加了一层硬件抽象层（Hardware Abstract Layer, HAL）。Android 引入硬件抽象层的主要原因可以归结成以下两个方面：

- Linux 宏内核属性使得设备驱动通常运行在内核态，内核驱动开发时不可避免地依赖内核驱动框架和各种子系统的支持，导致驱动实现与特定内核版本产生强耦合，阻碍了 Android 驱动生态的独立演进与升级。
- Linux 内核采用 GPLv2 开源协议，该协议具有“传染性”，这意味着运行在内核空间的设备驱动也必须一并开放源码。驱动开源会导致硬件设备的实现细节也被公开，一些设备厂商因此担心其硬件竞争力受到影响。

以手机厂商的拍照功能为例，拍照涉及相机设备以及 DSP、GPU、NPU 等加速器的图形渲染。好的拍照能力不仅需要配备高清镜头、光学传感器、闪光灯等硬件组件，还需要与之匹配的相机驱动和智能算法。相机驱动中包含相机设备实现的具体细节，智能算法则应用于相机控制中的图像预处理，用于实现自动聚焦、防抖动、运动跟踪等高级功能。高级的手机拍照功能甚至能过滤照片中的背景噪声，拍出夜空中群星璀璨的效果。无论是硬件细节还是智能算法，作为手机厂商的知识产权，厂商很可能不希望以 Linux 内核源码的形式公开，以此保证自身产品的市场竞争力。

为了促进更多的设备厂商加入 Android 生态，Android 操作系统设计了硬件抽象层，HAL 实现了 Linux 内核层与 Android 系统框架层的解耦，设备厂商不必开放源码就能为 Android 操作系统提供设备驱动以及相关的设备服务。Android 操作系统的具体架构如图 13.23 所示，在安卓系统服务和内核驱动之间架着一层 HAL。根据设备的具体功能和类型，Android 操作系统设计了相机、音频、指纹、触屏、显示、振动、NFC 等 HAL 模块，每种 HAL 模块都规定了一组对应设备类型的驱动所需实现的公共接口，以此封装不同设备厂商之间的实现细节。

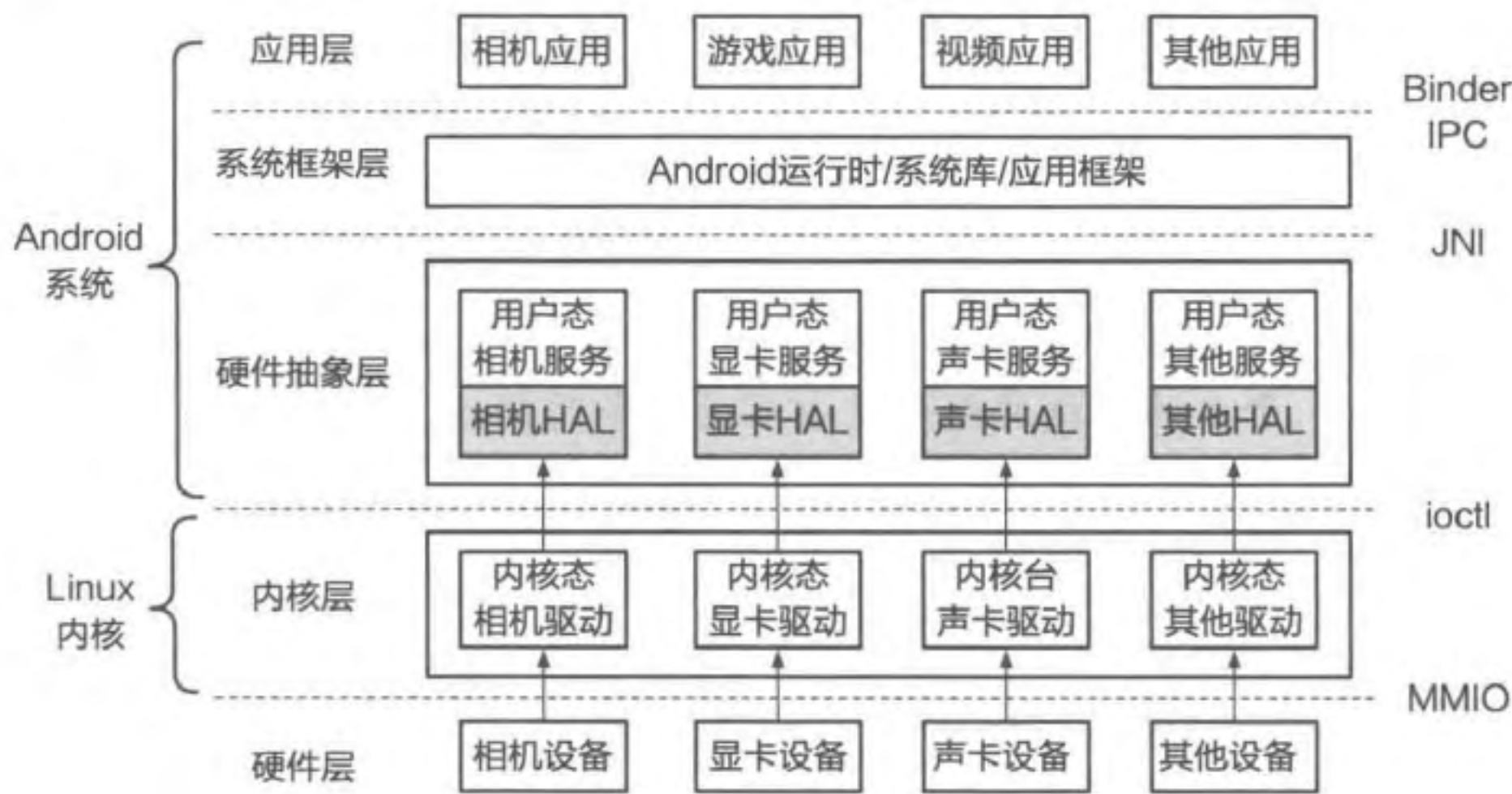


图 13.23 Android 操作系统在内核层之上增加一层硬件抽象层，从而实现 Android 驱动和内核驱动的解耦

HAL 层的用户态驱动使用 C++ 语言进行编写。Android 上层的系统框架层通过 JNI (Java Native Interface) 调用用户态驱动的 C++ 回调方法，而 HAL 用户态驱动则会打开下层的 Linux 设备文件，使用 ioctl 接口和底层的驱动进行通信。和 ChCore 等微内核操作系统一样，Android 操作系统的用户态驱动也编译为 .so 共享库的形式。和微内核操作系统的用户态驱动不同的是，HAL 用户态驱动依旧需要内核层的驱动支持，二者属于互补而非独立的关系。厂商可以根据需要将更多设备相关的操作放到用户态。

同样以相机设备的使用为例，Android 的相机应用会同时打开视图 (Surface) 和相机两个服务，前者获取相机摄像头的当前内容，后者允许用户操作相机本身。相机服务通

过 JNI 接口访问相机相关的硬件抽象层，即相机 HAL 接口。早期的相机 HAL 接口提供了预览、拍照和摄影三个基本功能，后续版本陆续增加了自动聚焦、自动曝光和自动白平衡以及后期处理等一系列高级拓展功能。来自内核层的 V4L2（Video for Linux 2）驱动负责图像帧的采集与转换等相对底层的能力。图 13.24 体现了这种分层设计。



图 13.24 Android 的相机 HAL 提供相机服务的相关功能，同时复用了底层 Linux 驱动的设备控制能力

小知识：用户态驱动的优势

尽管 Android 操作系统不是微内核操作系统，但是把设备驱动运行在用户态也符合微内核架构设计的思路。将驱动程序放到用户态，可以减少内核代码的体量，避免内核代码的膨胀，带来的直接好处是可以有效避免因驱动缺陷导致的内核崩溃问题。

除提高可靠性之外，用户态驱动还有如下优势：

- **开发灵活：**用户态驱动可以更加灵活地使用用户空间丰富的编程语言框架和共享库代码。
- **方便调试：**驱动程序以进程的形式存在，和系统服务以系统调用接口的形式进行交互，解耦更加彻底，调试起来也更为方便。
- **提高兼容性：**内核可能因为各种原因进行更新和重构，而系统调用作为系统服务接口通常更为稳定，在用户空间中实现驱动程序可以避免因为内核的变动而发生修改。

13.6 思考题

1. 持久化内存（Persistent Memory）或者非易失性内存（Non-volatile Memory）作为新型存储设备，既有内存的字节寻址能力，也有磁盘的持久化特性。如果请你来设计 PM/NVM 的驱动程序，你会将它归类为什么设备，并且如何最大限度地复用现有的系统